

# Documento de Arquitectura de Software (SAD) - Menu Bites

## 1. Resumen Ejecutivo

Menu Bites es una plataforma multitenant diseñada para la digitalización de menús y gestión de pedidos en tiempo real para restaurantes. El sistema permite a múltiples establecimientos (tenants) gestionar su inventario, mesas y pedidos de forma aislada y segura utilizando una arquitectura basada en microservicios gestionados (Supabase) y frontend reactivo (Next.js).

## 2. Objetivos Arquitectónicos

- Aislamiento Multitenant:** Garantizar que los datos de un restaurante sean inaccesibles para otros.
- Escalabilidad:** Uso de Serverless (Vercel) y BaaS (Supabase).
- Realtime:** Sincronización instantánea de pedidos mediante WebSockets.
- Offline Ready:** Capacidad básica de operación en la App Móvil.

## 3. Stack Tecnológico

- Frontend Web (Forkit Web):** Next.js 14 (App Router) + Tailwind CSS + Shadcn/UI.
- Frontend Mobile (Forkit Mobile):** React Native + Expo.
- Backend/Infra:** Supabase (PostgreSQL, Auth, Storage, Realtime).
- Validación:** Zod + Prisma (ORM para migraciones).

## 4. Diagramas de Arquitectura (Mermaid)

### 4.1 Diagrama de Contenedores (C4)

Visualización del sistema y sus interacciones con los actores principales.

```
graph TD
    User((Cliente/Garzón)) --> MobileApp[App Móvil - Expo]
    Admin((Administrador)) --> WebApp[App Web - Next.js 14]
    MobileApp --> Supabase[Supabase Platform]
    WebApp --> Supabase
    subgraph Supabase
        Auth[Supabase Auth]
        DB[(PostgreSQL)]
        Storage[Supabase Storage]
        Realtime[Realtime Engine]
    end
    end
    DB --> Realtime
```

### 4.2 Modelo de Entidad-Relación (ERD)

Estructura de datos optimizada para el aislamiento tenant.

```
erDiagram
    RESTAURANT ||--o{ CATEGORY : manages
    RESTAURANT ||--o{ TABLE : contains
```

```
CATEGORY ||--o{ MENU_ITEM : includes
TABLE ||--o{ ORDER : places
ORDER ||--o{ ORDER_ITEM : has
MENU_ITEM ||--o{ ORDER_ITEM : ordered
```

### 4.3 Ciclo de Vida del Pedido (Máquina de Estados)

Estados críticos para la consistencia de la operación.

```
stateDiagram-v2
    [*] --> PENDING
    PENDING --> VALIDATED : Confirmación
    VALIDATED --> PREPARING : En Cocina
    PREPARING --> READY : Plato Terminado
    READY --> DELIVERED : Entrega a Mesa
    DELIVERED --> [*]
```

### 4.4 Flujo de Autenticación y Redirección (Auth Proxy)

Lógica de acceso basada en roles y tenant\_id.

```
sequenceDiagram
    participant U as Usuario
    participant P as Proxy Middleware
    participant S as Supabase Auth
    participant D as Operative Dashboard

    U->>P: Request Page (e.g. /cashier)
    P->>S: getUser() (Session Validation)
    alt No Session
        S-->>P: No User
        P-->>U: Redirect to Login (/)
    else Valid Session
        S-->>P: User Data + app_metadata
        P->>P: Check role & restaurant_id
        alt SuperAdmin
            P-->>U: Proceed to /admin
        else Operative Role (GARZON/CAJERO/COCINA)
            P->>P: Map Role to Target URL
            P-->>D: Proxy/Redirect to Dashboard
            D-->>U: Render Interface
        end
    end
end
```

### 4.5 Modelo de Aislamiento de Datos (RLS)

Garantía inquebrantable de privacidad entre restaurantes.

```
graph LR
    User((User((Usuario Autenticado))) -- JWT --> Supabase((Supabase Engine))
```

```
Supabase -- get_auth_restaurant_id() --> RLS{Row Level Security}
subgraph PostgreSQL
  Table[(Tablas Compartidas)]
  RLS -- tenant_id = current_setting --> Table
end
Table -- Filas Filtradas --> User
```

## 5. Decisiones Arquitectónicas (ADRs)

1. **Uso de Next.js Proxy:** Para centralizar la autenticación y evitar CORS en subdominios locales.
2. **Supabase Realtime:** Elegido por sobre Socket.io para reducir la complejidad de infraestructura.
3. **RLS Basado en Claims:** Permite que el aislamiento ocurra a nivel de base de datos, no de aplicación, eliminando errores de filtrado manual.